Ekstraksi fitur, Seleksi Fitur, & Reduksi Dimensi

Ir. Fandy Setyo Utomo, S.Kom., M.Cs., Ph.D

PSIFW074 – Pembelajaran Mesin

Program Studi Informatika - Universitas AMIKOM Purwokerto

Capaian Pembelajaran Praktikum:

Mahasiswa mampu membangun model ekstraksi fitur, seleksi fitur, dan reduksi dimensi untuk menyelesaikan permasalahan data.

Software yang diperlukan dalam pembelajaran:

1. Python
2. Pandas (https://pandas.pydata.org/)
3. Seaborn (https://seaborn.pydata.org/)
4. Matplotlib (https://matplotlib.org/)
5. Scikit-learn/sklearn (https://scikit-learn.org/stable/)
6. Spyder (*Scientific Python Development Environment*)

Tahapan Praktikum:

1. Import seluruh pustaka yang dibutuhkan untuk operasi data pre-processing pada IDE Spyder:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
```

Catatan:

- *Matplotlib* merupakan pustaka untuk membuat visualisasi statis, animasi, dan interaktif di dalam lingkungan Python.
- *Seaborn* merupakan pustaka untuk visualisasi data pada lingkungan pemrograman Python berbasis Matplotlib. Pustaka ini menyediakan antarmuka tingkat tinggi untuk menggambar grafik statistik yang menarik dan informatif.

2. Import dataset dengan menuliskan perintah berikut pada IDE Spyder:

```
#import dataset
dataset=pd.read_csv('california_housing_dataset.csv')
df=pd.DataFrame(dataset)
```

Amati isi variabel ***df*** pada bagian *variable explorer*! Berikut penjelasan dari masing-masing atribut datanya (*feature variables*):

- ***longitude***: Geographic longitude of the house location.
- ***latitude***: Geographic latitude of the house location.
- ***housing_median_age***: Median age of houses in the area.
- ***total_rooms***: Total number of rooms in the area.
- ***total_bedrooms***: Total number of bedrooms in the area.
- ***population***: Total population in the area.
- ***households***: Number of households in the area.
- ***median_income***: Median income of households in the area.
- ***ocean_proximity***: Proximity of the houses to the ocean.
- ***median_house_value***: median house price

median_house_value menjadi target variabel dalam model prediksi harga rumah.

3. Identifikasi *missing values* dengan menuliskan perintah berikut pada IDE Spyder:

```
1. #identifikasi missing values
2. print(df.info())
3. mv=df.isna().sum()
4. print('\nJumlah missing value tiap kolom:\n', mv)
```

Catatan:

- Hasil eksekusi kode program pada baris ke-4:

```
Jumlah missing value tiap kolom:
 longitude               0
latitude                 0
housing_median_age       0
total_rooms              0
total_bedrooms         207
population               0
households               0
median_income            0
median_house_value       0
ocean_proximity          0
dtype: int64
```

Gambar 1. Jumlah *Missing Value* Tiap Kolom

Berdasarkan output yang diberikan pada *IPython Console*, tampak bahwa kolom ***total_bedrooms*** memiliki *missing value* sejumlah 207 data.

4. Permasalahan *missing value* pada kolom ***total_bedrooms*** ditangani dengan mengisi *value NaN* dengan nilai median. Nilai median diperoleh dengan 2 tahapan, yaitu: (1) Membagi dataframe menjadi 20 blok berdasarkan value pada kolom ***households***; dan (2) Mengisi nilai NaN pada kolom *total_bedrooms* dengan median dari blok *total_bedrooms* tempat record data tersebut berasal.

   Solusi ini dapat diimplementasikan dengan menuliskan perintah berikut pada IDE Spyder:

```
'''
Solusi missing value:
Isi missing value dengan nilai median
'''
#membagi dataframe menjadi 20 blok berdasarkan 'households'
df['block'] = pd.cut(df['households'], bins=20, labels=False)

#fungsi untuk mengisi NaN dengan median dari setiap blok
def fill_nan_with_block_median(group):
    return group.fillna(group.median())

#mengisi NaN pada kolom 'total_bedrooms' dengan median dari
#blok 'total_bedrooms' dari blok record tersebut berasal
df['total_bedrooms'] =
df.groupby('block')['total_bedrooms'].transform(fill_nan_with_block_median)

#menghapus kolom 'block' setelah proses pengisian NaN selesai
df.drop(columns=['block'], inplace=True)
```

   Catatan:

   - **pd.cut(...)** digunakan *cut* ketika diperlukan pengelompokkan dan pengurutan nilai data ke dalam *bins*. Fungsi ini juga berguna untuk beralih dari variabel kontinu ke variabel kategorikal.
   - Amati isi variabel DataFrame ***df*** pada variable explorer! Apa perbedaannya dengan nilai ***df*** yang sebelumnya?

Latihan:

Ubah nilai *NaN* (*missing value*) menjadi nilai Mean atau nilai yang paling sering muncul (*most frequent*)!

5. Penanganan variabel *ocean_proximity* sebagai permasalahan ***Categorical Data*** dapat dikodekan menggunakan ***LabelEncoder*** dengan instruksi sebagai berikut:

```
'''
Solusi Categorical Data:
Pengodean kolom ocean_proximity dengan LabelEncoder
'''
label_encoder_x= LabelEncoder()
df['ocean_proximity']= label_encoder_x.fit_transform(df['ocean_proximity'])
```

Amati isi variabel ***df***. Apa perbedaan antara isi variabel tersebut saat ini dengan sebelumnya?

6. Identifikasi korelasi antar atribut di dalam *DataFrame* dengan menuliskan perintah berikut pada IDE Spyder:

```
#identifikasi korelasi antar variabel
correlation = df.corr(method='pearson')
```

Catatan:

- **df.corr(method='pearson')** digunakan untuk menghitung korelasi kolom secara berpasangan, tidak termasuk nilai NaN/null. Dalam studi kasus ini metode "*Pearson correlation*" digunakan untuk menghitung korelasi antar atribut / variabel. Informasi lebih detil dapat disimak pada website berikut ini: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.corr.html
- Amati isi variabel *correlation* pada bagian *variable explorer*:

correlation - DataFrame

Index	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
longitude	1	-0.924664	-0.108197	0.044568	0.069608	0.0997732	0.0553101	-0.0151759	-0.0459666
latitude	-0.924664	1	0.0111727	-0.0360996	-0.0669828	-0.108785	-0.0710354	-0.0798091	-0.14416
housing_median_age	-0.108197	0.0111727	1	-0.361262	-0.320451	-0.296244	-0.302916	-0.119034	0.105623
total_rooms	0.044568	-0.0360996	-0.361262	1	0.93038	0.857126	0.918484	0.19805	0.134153
total_bedrooms	0.069608	-0.0669828	-0.320451	0.93038	1	0.877747	0.979728	-0.00772285	0.0496862
population	0.0997732	-0.108785	-0.296244	0.857126	0.877747	1	0.907222	0.00483435	-0.0246497
households	0.0553101	-0.0710354	-0.302916	0.918484	0.979728	0.907222	1	0.0130331	0.0658427
median_income	-0.0151759	-0.0798091	-0.119034	0.19805	-0.00772285	0.00483435	0.0130331	1	0.688075
median_house_value	-0.0459666	-0.14416	0.105623	0.134153	0.0496862	-0.0246497	0.0658427	0.688075	1

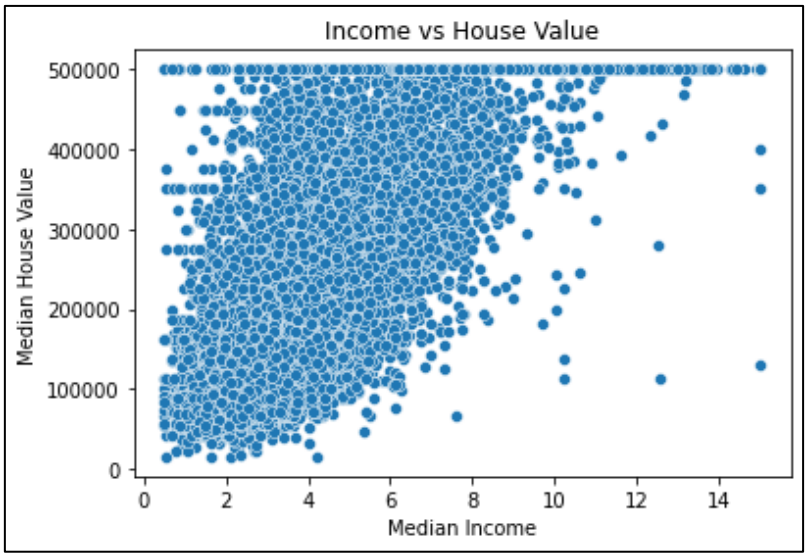
Gambar 2. Variabel *correlation*

Berdasarkan Gambar 2, dapat diperhatikan bahwa terdapat korelasi yang tinggi antara atribut ***total_bedrooms*** dengan ***households*** dengan nilai **0.979728**.

7. Berdasarkan Gambar 2, *median_income* merupakan variabel yang paling berkorelasi dengan *median_house_value* dengan koefisien korelasi sebesar 0,688075. Scatter plot untuk korelasi kedua variabel tersebut dapat diimplementasikan dengan instruksi sebagai berikut:

```
#scatter plot:'median_income' vs 'median_house_value'
sns.scatterplot(x='median_income', y='median_house_value', data=df)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Income vs House Value')
plt.show()
```

Amati output hasil eksekusi kode program tersebut yang ditunjukkan pada bagian *IPython Console*.



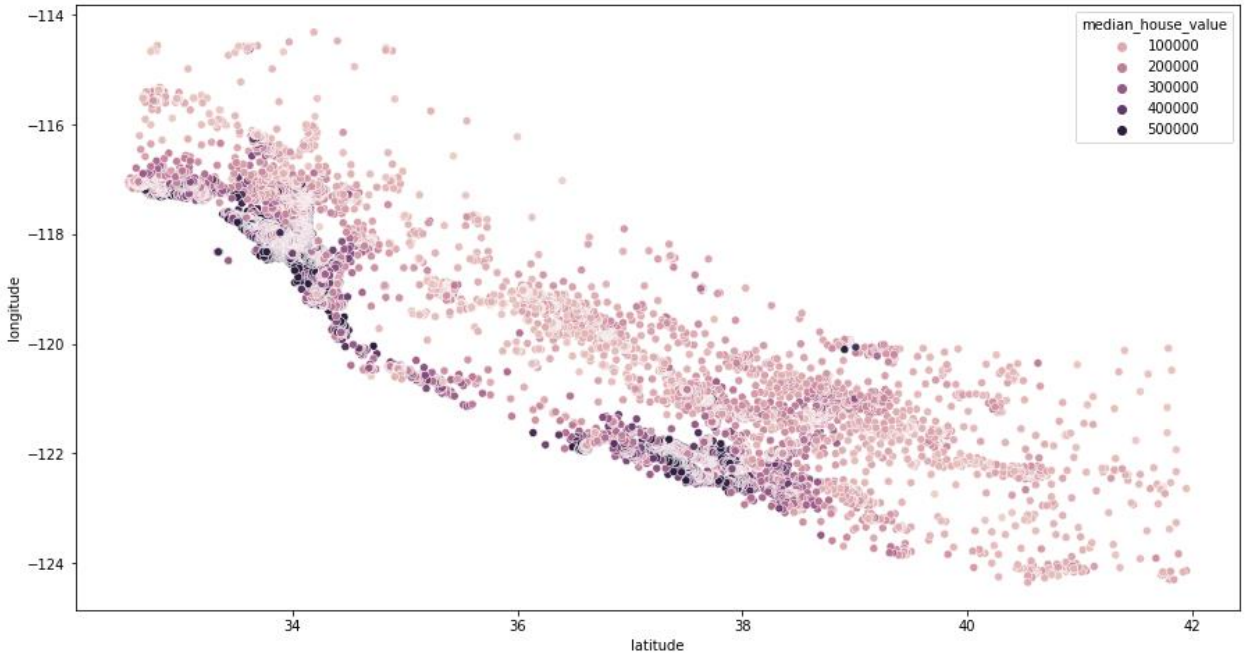
Gambar 3. Scatter Plot Median Income vs Median House Value

Apa yang dapat anda simpulkan dari Gambar 3 tersebut?

8. Instruksi berikut ini digunakan untuk mendeskripsikan citra Scatter plot antara variabel *latitude* dan *longitude*:

```
#scatter plot: 'latitude' & 'longitude'
plt.figure(figsize=(15,8))
sns.scatterplot(x='latitude', y='longitude', data=df, hue='median_house_value')
```

Amati output hasil eksekusi kode program tersebut yang ditunjukkan pada bagian *IPython Console*.



Gambar 4. Scatter Plot Latitude vs Longitude

Apa yang dapat anda simpulkan dari Gambar 4 tersebut?

9. Penanganan *feature scaling* pada variabel *latitude* dan *longitude* menggunakan ***MinMaxScaler***. Menurut pendapat anda, mengapa kedua variabel tersebut perlu dilakukan *feature scaling*? Implementasi *feature scaling* pada studi kasus ini menggunakan ***MinMaxScaler*** yang dituliskan dalam instruksi berikut:

```
'''
Solusi feature scaling dengan MinMaxScaler
'''
scaler = MinMaxScaler()
df[['latitude', 'longitude']] = scaler.fit_transform(df[['latitude', 'longitude']])
```

10. Selanjutnya, ekstraksi variabel independen dan dependen dengan instruksi sebagai berikut:

```
#ekstraksi variabel independen
X = df.drop(columns=['median_house_value'])

#ekstraksi variabel dependen
y = df['median_house_value']
```

Apa perbedaan antara variabel independen dan dependen? Amati variabel ***X*** dan ***y*** pada bagian *variable explorer*! Apa perbedaan antara kedua variabel tersebut?

11. Seleksi fitur pada studi kasus ini menggunakan model *Chi-Square* dengan instruksi sebagai berikut:

```
'''
seleksi fitur menggunakan chi-square
'''
# Memilih 7 fitur terbaik
selector = SelectKBest(score_func=chi2, k=7)
X_selected = selector.fit_transform(X, y)

#mendapatkan indeks fitur yang tereliminasi
eliminated_features_index = ~selector.get_support()

#mendapatkan nama fitur yang tereliminasi
eliminated_features = X.columns[eliminated_features_index]

#menampilkan fitur-fitur yang tereliminasi
print("Fitur-fitur yang tereliminasi:")
print(eliminated_features)
```

Catatan:

- Apa fitur yang tereliminasi?
- **SelectKBest(...)** digunakan untuk memilih fitur berdasarkan perankingan nilai ***k***. Informasi lebih detil mengenai method ini dapat diakses melalui website: https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html
- Model feature selection lainnya dapat anda pelajari melalui website berikut ini: https://scikit-learn.org/stable/modules/classes.html#module-sklearn.feature_selection

Eksperimen:
Gunakan minimal 3 model seleksi fitur lainnya pada studi kasus ini dan amati output yang diberikan!

12. Langkah terakhir, bagi dataset menjadi data latih (*training data*) dan data uji (*testing data*) dengan instruksi sebagai berikut:

```
'''
Data spliting
'''
#pembagian dataset menjadi training dan test set
X_train, X_test, y_train, y_test = train_test_split(X_selected, y, test_size = 0.2,
random_state = 0)
```

Perhatikan isi variabel ***X_train, X_test, y_train, y_test*** pada window Variable Explorer! Apa kesimpulan anda?

Latihan:

Ubah ukuran data latih menjadi 85%!

KODE PROGRAM GABUNGAN

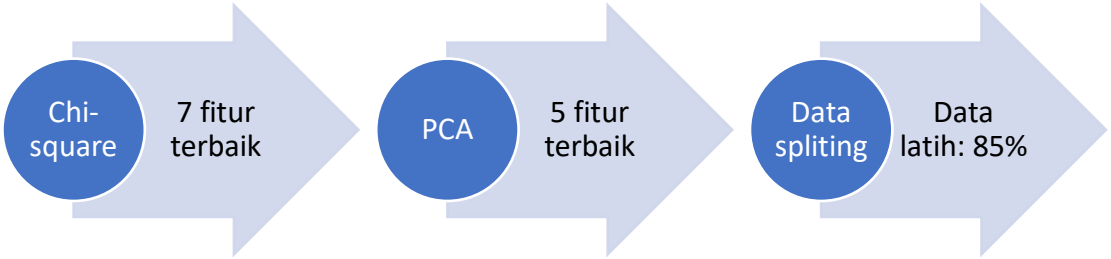
Berikut ini merupakan kode program gabungan, mulai dari import pustaka yang dibutuhkan untuk menyelesaikan studi kasus hingga pembagian dataset.

```
#import libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest, chi2
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split

#import dataset
dataset=pd.read_csv('california_housing_dataset.csv')
df=pd.DataFrame(dataset)

#identifikasi missing values
print(df.info())
mv=df.isna().sum()
print('\nJumlah missing value tiap kolom:\n', mv)

'''
Solusi missing value:
Isi missing value dengan nilai median
'''
#membagi dataframe menjadi 20 blok berdasarkan 'households'
df['block'] = pd.cut(df['households'], bins=20, labels=False)

#fungsi untuk mengisi NaN dengan median dari setiap blok
def fill_nan_with_block_median(group):
    return group.fillna(group.median())

#mengisi NaN pada kolom 'total_bedrooms' dengan median dari
#blok 'total_bedrooms' dari blok record tersebut berasal
```

```
df['total_bedrooms'] = df.groupby('block')['total_bedrooms'].transform(fill_nan_with_block_median)

#kode bantu untuk cek blok dan median secara manual
block_0_records = df.loc[df['block'] == 0]
median_households_block_0 = block_0_records['total_bedrooms'].median()
print("Nilai median kolom 'total_bedrooms' pada block 0:", median_households_block_0)

#menghapus kolom 'block' setelah proses pengisian NaN selesai
df.drop(columns=['block'], inplace=True)

'''
Solusi Categorical Data:
Pengodean kolom ocean_proximity dengan LabelEncoder
'''
label_encoder_x= LabelEncoder()
df['ocean_proximity']= label_encoder_x.fit_transform(df['ocean_proximity'])

#identifikasi korelasi antar variabel
correlation = df.corr(method='pearson')

#scatter plot:'median_income' vs 'median_house_value'
sns.scatterplot(x='median_income', y='median_house_value', data=df)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Income vs House Value')
plt.show()

#scatter plot: 'latitude' & 'longitude'
plt.figure(figsize=(15,8))
sns.scatterplot(x='latitude', y='longitude', data=df, hue='median_house_value')

'''
Solusi feature scaling dengan MinMaxScaler
'''
scaler = MinMaxScaler()
df[['latitude', 'longitude']] = scaler.fit_transform(df[['latitude', 'longitude']])

#ekstraksi variabel independen
X = df.drop(columns=['median_house_value'])

#ekstraksi variabel dependen
y = df['median_house_value']

'''
seleksi fitur menggunakan chi-square
'''
# Memilih 7 fitur terbaik
selector = SelectKBest(score_func=chi2, k=7)
X_selected = selector.fit_transform(X, y)

#mendapatkan indeks fitur yang tereliminasi
eliminated_features_index = ~selector.get_support()

#mendapatkan nama fitur yang tereliminasi
eliminated_features = X.columns[eliminated_features_index]

#menampilkan fitur-fitur yang tereliminasi
print("Fitur-fitur yang tereliminasi:")
print(eliminated_features)

'''
Data spliting
'''
#pembagian dataset menjadi training dan test set
X_train, X_test, y_train, y_test = train_test_split(X_selected, y, test_size = 0.2, random_state = 0)
```

Tambahan Soal Latihan!

Selain menggunakan metode *feature selection* seperti *chi-square*, gunakan pula reduksi dimensi dengan metode ***Principal Component Analysis*** (PCA) untuk mengeliminasi fitur yang memiliki pengaruh/nilai terendah. Ilustrasi alurnya sebagai berikut:



Laporan Praktikum:

Laporan praktikum yang berisi modifikasi kode program yang telah diubah berdasarkan latihan praktikum diunggah melalui halaman: https://forms.gle/znCaFz4R71FDKVfX7 paling lambat H-1 sebelum perkuliahan minggu depan. Kolom "Tugas Pertemuan" pilih pertemuan "3."